

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Karpaga shree

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for(i = 0; i <= n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1,2,3,4,5}
    int total;
    total = sumArray(arr);

    if(total == 15) {

        printf("Correct Sum\n")
    }

    int a = 10, b = 0;
    int c = a / b;

    int *p;
    *p = 20;

    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
```

```
int main() {  
    int lnum = 100;  
    float rate@ = 25.5;  
    char ch = 'abc';  
}
```

```

int total = 50$20;
int num#1 = 30;
int %count = 40;
printf("Welcome to Compiler Design);
int x = 09;
int y = 0xZ12;
int z = 12.3.4;

char str1[] = "Hello;
char str2[] = "World";

int _valid = 10;
int invalid-id = 20;

float value = 5..6;
int data = 10@5;

char c = 'xy';
int 3value =
60;

printf("Value: %d\n", total);

return 0;
}

```

Tasks

- (a) Error Identification
 - i) Identify all lexical errors in the program.
 - ii) List the line number and the invalid token.
- (b) Error Classification
 - i) For each error, explain why it is considered a lexical error.
 - ii) Specify which rule of token formation is violated:
 - iii) Identifier rules
 - iv) Constant formation
 - v) String/character literal rules
 - vi) Operator/symbol rules
- (c) Token Correction
 - i) Rewrite each invalid token into a valid token.
 - ii) Example: 2value → value2
- (d) Compiler Behavior Analysis
 - i) Explain how a lexical analyzer (scanner) processes such errors.
 - ii) Discuss whether the compiler:
 - iii) Stops immediately
 - iv) Continues scanning using error recovery
- (e) Advanced Thinking
 - i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
 - ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

- A. Lexical Errors
- B. Syntax Errors
- C. Semantic Errors
- D. Intermediate Code Generation Issues (logic/precedence)
- E. Code Optimization Issues (redundancy/inefficiency)
- F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>
```

```
int divide(int a, int b) {  
    return a / b  
}
```

```
int main() {  
    int #value = 20;  
    float num = 15.5  
    int result;  
    result = divide(10);
```

```
    if(result = 5) {  
  
        printf("Result is 5\n")  
    }
```

```
    int x = 10;  
    int y = 0;  
    int z = x / y;
```

```
    int arr[4] = {1,2,3,4};  
    printf("%d\n", arr[4]);
```

```
    int *ptr;  
    *ptr = 25;
```

```
    int total;  
    total = x + y * result + 0;
```

```
    while(x < 20) {
```

```
    printf("%d\n", x);  
}  
  
return 0;  
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

- I. Identify **all errors** in the program.
- II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

- I. Rewrite the **fully corrected program**.
- II. Ensure:
 - A. Proper syntax
 - B. Correct semantics
 - C. Safe execution

(e) Deep Analysis (Evaluation)

- I. Explain how **incorrect pointer usage** affects runtime.
- II. Analyze how **array bounds violation** impacts memory.
- III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

- I. Suggest **2 improvements** in compiler design to:
 - A. Detect runtime errors earlier
 - B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Compiler Design (CSA1407) - Assessment Tool 2

Error Analysis Task - Complete Solution

Question 1

The given C program performs array summation through a function call, checks the result, and then performs additional arithmetic and pointer operations. The program has been analyzed line by line to detect errors introduced at the lexical, syntax, semantic, logical, and runtime levels.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int total;
    int n =
    5;
    total = sumArray(arr, n);
    if (total == 15) {
        printf("Correct Sum\n");
    }
    int a = 10, b = 2;
    int c = a / b;
    printf("Value of c = %d\n", c);
    int p_val = 20;
    int *p = &p_val;
    *p = 20;
    printf("Value pointed by p = %d\n", *p);
    return 0;
}
```


(a) Error Identification (Analysis Level)

S.No.	Line No.	Error Description
1	12	Missing semicolon after the array initialization statement 'int arr[5] = {1,2,3,4,5}'.
2	14	Function sumArray() is defined to accept two parameters (arr, n) but is called with only one argument, 'arr'.
3	5	Loop condition uses 'i <= n' instead of 'i < n'. Since the array has 5 valid indices (0–4), this causes an out-of-bounds access at arr[n].
4	16	Missing semicolon after the statement 'printf("Correct Sum\n")'.
5	19	Variable 'b' is assigned the value 0, and used as a divisor in 'a / b', causing division by zero.
6	20–21	Pointer 'p' is declared but never initialized to a valid memory address before being dereferenced in '*p = 20'.

(b) Classification of Errors (Understanding + Analysis)

Line No.	Error	Compiler Phase
12	Missing semicolon after array initialization	Syntax Error
16	Missing semicolon after printf statement	Syntax Error
14	Function called with fewer arguments than declared (argument mismatch)	Semantic Error
5	Loop runs one extra time (off-by-one boundary condition)	Logical Error
19	Division of an integer by zero	Runtime Error
20–21	Dereferencing an uninitialized (wild) pointer	Runtime Error

No lexical errors (invalid tokens, illegal symbols, or malformed constants) are present in this particular program; all identifiers, keywords, and constants are correctly formed at the character level.

(c) Compilation Behavior (Evaluation Level)

Errors detected during compilation: The missing semicolons on lines 12 and 16 are syntax errors and are always caught by the compiler's parser, since a valid statement terminator is mandatory in C. The argument-count mismatch on line 14 is also detected at compile time by modern compilers such as GCC, because the function prototype for sumArray() is visible before its call, allowing the type checker to verify the number of parameters.

Errors detected only during execution: The off-by-one loop condition (line 5), the division by zero (line 19), and the uninitialized pointer dereference (lines 20–21) are not flagged by the

compiler because they are syntactically and (in the case of the loop and pointer) semantically legal C statements. Their incorrect behavior — reading `arr[5]`, triggering a SIGFPE arithmetic exception, or writing to an unknown memory address — only becomes apparent when the program is actually run.

How early-stage errors mask later errors: When the parser encounters the missing semicolon on line 12, it cannot correctly determine where the declaration statement ends. This forces the compiler into panic-mode error recovery, where it discards tokens until it finds a synchronization point such as the next semicolon or brace. As a result, the next line (13) may be wrongly merged with line 12 in the parse tree, producing confusing or cascading syntax error messages (for example, 'expected ";" before "int"'). Because semantic analysis and later phases (intermediate code generation, optimization, code generation) only run on a syntactically valid parse tree, the compiler cannot proceed to check deeper issues — such as the argument mismatch on line 14 — until every syntax error is resolved. This is why syntax errors are always reported and fixed before semantic, logical, or runtime errors can even be evaluated.

(d) Code Correction (Application Level)

The corrected program below removes every identified error: semicolons are inserted, the function call passes both required arguments, the loop condition is fixed to avoid out-of-bounds access, the divisor is changed to a non-zero value, and the pointer is initialized to point to a valid memory location before being dereferenced.

```
#include <stdio.h>
int sumArray(int arr[], int n)
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
    {
        sum = sum +
        arr[i];
    }
    return sum;
}
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int total;
    total = sumArray(arr, 5);
    if (total == 15) {
        printf("Correct Sum\n");
    }
    int a = 10, b = 2;
    int c = a / b;
    printf("Value of c = %d\n", c);
    int value = 0;
    int *p = &value;
    *p = 20;
```

```
    printf("Value pointed by p = %d\n", *p);  
    return 0;  
}
```

Expected Output:

Correct Sum

Value of c = 5

Value pointed by p = 20

(e) Impact Analysis (Evaluation Level)

1. Off-by-one loop condition (line 5): Using 'i <= n' makes the loop read arr[5], one element past the last valid index (arr[0] to arr[4]). In C, arrays do not perform automatic bounds checking, so this reads whatever value happens to occupy adjacent memory. The impact is a silently incorrect sum — the program does not crash but produces an unreliable result, which is often more dangerous than an outright crash because the error can go unnoticed.
2. Division by zero (line 19): Integer division by zero is undefined behavior in C. On most systems it raises a SIGFPE (arithmetic exception) signal, which immediately terminates the program. This shows how a single unchecked arithmetic operation can abruptly halt an otherwise working program, which is why input validation before division is essential in real-world software.
3. Improper pointer usage (lines 20–21): Dereferencing an uninitialized pointer writes the value 20 to whatever random memory address happens to be stored in 'p'. This can corrupt unrelated data, overwrite another program's memory, or immediately trigger a segmentation fault. Such wild-pointer bugs are a leading cause of security vulnerabilities and are notoriously difficult to reproduce and debug because the effect depends on the arbitrary memory address involved.
4. General effect of careless loop control (infinite loops): Although this specific program terminates, the same class of mistake seen in the off-by-one condition — an incorrect loop-termination test — can, if the update expression is also wrong (for example, forgetting to increment the counter), create an infinite loop. This causes the program to hang indefinitely, continuously consuming CPU cycles and memory, and in server-side or embedded systems can lead to complete system unresponsiveness or denial of service.

(f) Advanced Compiler Perspective (Create Level)

Better error messages: A compiler can improve diagnostics by pointing to the exact character/column where a rule is broken (caret-style diagnostics, as in Clang), explaining the rule in plain language, and suggesting a concrete fix — for example, instead of a generic 'syntax error', showing 'Error at line 12: missing ";" after array initializer — did you forget a semicolon?'. For semantic errors, messages can name the exact mismatch, such as 'sumArray() expects 2 arguments but 1 was supplied; the parameter "n" is missing.'

Early detection of runtime errors: Many runtime errors can be caught earlier using static analysis integrated into the compiler front end: definite-assignment analysis to flag uninitialized pointers/variables before they are used, symbolic range analysis on loop bounds to detect off-by-one array accesses, and simple constant-propagation checks to warn when a divisor is a compile-time-known zero.

Two proposed enhancements: (1) Integrate a data-flow analysis pass between the semantic-analysis and intermediate-code-generation phases that tracks the initialization and validity state of every pointer and array index, issuing compile-time warnings for likely wild-pointer or out-of-bounds usage. (2) Adopt panic-mode-plus-suggestion error recovery, where the compiler continues scanning after an error (instead of stopping at the first one), collects all errors from a single pass, and pairs each with an auto-suggested correction — significantly reducing the number of compile-fix-recompile cycles needed for debugging.

Question 2

The following C program contains multiple lexical errors introduced intentionally through invalid tokens, incorrect identifiers, malformed constants, and illegal symbols — all of which are detected during the lexical analysis (scanning) phase of a compiler.

Given Program

```
#include <stdio.h>
int main() {
    int 1num = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;
    printf("Welcome to Compiler Design);
    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;
    char str1[] = "Hello;
    char str2[] = "World";
    int _valid = 10;
    int invalid-id = 20;
    float value = 5..6;
    int data = 10@5;
    char c = 'xy';
    int 3value = 60;
    printf("Value: %d\n", total);
    return 0;
}
```

(a) Error Identification

All sixteen lexical errors, with their line numbers and the exact invalid token, are listed below.

S.No.	Line	Invalid Token	Nature of Error
1	3	1num	Identifier begins with a digit
2	4	rate@	Illegal symbol '@' inside an identifier
3	5	'abc'	Character constant contains more than one character
4	6	50\$20	Illegal symbol '\$' inside a numeric constant
5	7	num#1	Illegal symbol '#' inside an identifier
6	8	%count	Identifier begins with the illegal symbol '%'
7	9	"Welcome to Compiler Design)	Unterminated string literal (missing closing quote)
8	10	09	Invalid octal constant (digit 9 not allowed after leading 0)
9	11	0xZ12	Invalid hexadecimal constant ('Z' is not a hex digit)
10	12	12.3.4	Malformed floating-point constant (two decimal points)
11	13	"Hello;	Unterminated string literal (missing closing quote)
12	16	invalid-id	Hyphen not permitted in an identifier
13	17	5..6	Malformed floating-point constant (two consecutive decimal points)
14	18	10@5	Illegal symbol '@' inside a numeric expression
15	19	'xy'	Character constant contains more than one character
16	20	3value	Identifier begins with a digit

Lines 14 ("World") and 15 (_valid) are correctly formed and are not errors; they are included in the program only for contrast with the invalid tokens around them.

(b) Error Classification

Each invalid token is a lexical error because it is rejected at the character/token-scanning stage itself — before any parsing, meaning checking, or execution can take place. The scanner cannot map the character sequence to any valid token defined by the language's regular-expression rules, so the failure occurs at the earliest phase of compilation.

Rule Violated	Lines / Tokens Affected
Identifier rules (must start with a letter or underscore; may contain only letters, digits, underscore)	3 (1num), 4 (rate@), 7 (num#1), 8 (%count), 16 (invalid-id), 20 (3value)
Constant formation rules (integer, octal, hex, and floating-point constants must follow fixed digit patterns)	10 (09), 11 (0xZ12), 12 (12.3.4), 17 (5..6)
String / character literal rules (a string must be closed with a matching quote; a char constant must hold exactly one character)	5 ('abc'), 9 (unterminated string), 13 (unterminated string), 19 ('xy')
Operator / symbol rules (only defined operators and symbols may appear in expressions)	6 (50\$20), 18 (10@5)

(c) Token Correction

Line	Invalid Token	Corrected Token
3	1num	num1
4	rate@	rate
5	'abc'	'a' (or use a string: char ch[] = "abc";)
6	50\$20	5020
7	num#1	num1
8	%count	count
9	"Welcome to Compiler Design)	"Welcome to Compiler Design"
10	09	9
11	0xZ12	0x112
12	12.3.4	12.34
13	"Hello;	"Hello"
16	invalid-id	invalid_id
17	5..6	5.6
18	10@5	105
19	'xy'	'x'
20	3value	value3

(d) Compiler Behavior Analysis

How the lexical analyzer processes such errors: The scanner reads the source program character by character and groups characters into the longest matching lexeme using the regular-expression / DFA rules defined for each token class (identifiers, keywords, constants, operators, punctuation). When the current character cannot extend any valid token pattern — for example, a digit appearing as the first character of what should be an identifier, or a special symbol such as '@', '#', or '\$' appearing where no operator is defined — the scanner reports an 'invalid token' or 'unrecognized character' error at that exact position.

Does the compiler stop immediately or continue? A well-designed lexical analyzer does not stop at the first error. It uses panic-mode error recovery: it discards the offending character(s) and resumes scanning from the next character, or skips ahead to the next delimiter such as whitespace, a semicolon, or a newline, and continues building the token stream. This is why a compiler such as GCC can report all sixteen lexical errors in this program in a single compilation pass rather than stopping after the very first one (1num on line 3). Only if the scanner cannot re-synchronize at all does compilation abort early.

(e) Advanced Thinking

DFA idea to detect identifiers starting with a digit: A minimal DFA for validating identifiers uses three states — q0 (start), q1 (valid identifier, accepting), and qE (dead/error state).

Current State	On Letter or ' _ '	On Digit	On Any Other Symbol
q0 (start)	→ q1	→ qE	→ qE
q1 (accepting)	→ q1	→ q1	→ stop / accept token
qE (dead state)	→ qE	→ qE	→ qE

Rule: the very first character determines whether the DFA moves from q0 to the accepting state q1 (letter or underscore) or to the dead state qE (digit or illegal symbol). If the DFA reaches qE, the lexer immediately reports 'Invalid Identifier: cannot start with a digit' rather than continuing to build the token, since no identifier lexeme can ever be accepted once the automaton has entered the dead state.

Improving error messages for beginners: Error messages can be made far more approachable by (1) quoting the exact invalid token and underlining the offending character with a caret, (2) stating the specific rule that was broken in plain English rather than compiler jargon, (3) suggesting a corrected version of the token (as shown in the token-correction table above), and

(4) grouping related errors together (for example, all identifier-naming errors under one heading) so that a beginner is not overwhelmed by a long, undifferentiated list of unrelated messages.

Question 3

The following program performs arithmetic, a function call, a conditional check, array access, pointer usage, and looping. It contains errors distributed across every phase of the compiler: lexical, syntax, semantic, intermediate code generation, code optimization, and runtime / target code.

Given Program (as provided)

```
#include <stdio.h>
int divide(int a, int b) {
    return a / b
}
int main() {
    int #value = 20;
    float num = 15.5
    int result;
    result = divide(10);
    if(result = 5) {
        printf("Result is 5\n")
    }
    int x = 10;
    int y = 0;
    int z = x /
    y;
    int arr[4] = {1,2,3,4};
    printf("%d\n",
    arr[4]); int *ptr;
    *ptr = 25;
    int total;
    total = x + y * result + 0;
    while(x < 20) {
        printf("%d\n", x);
    }
    return 0;
}
```

(a) Identify Errors (Analysis Level)

S.No.	Line No.	Error Description
1	8	Identifier '#value' contains the illegal symbol '#'.
2	4	Missing semicolon after 'return a / b'.

3	9	Missing semicolon after the declaration of 'num'.
---	---	---

S.No.	Line No.	Error Description
4	11	Function divide() requires two arguments (a, b) but is called with only one, '10'.
5	12	Assignment operator '=' used instead of the comparison operator '==' inside the if condition.
6	13	Missing semicolon after the printf statement.
7	17	Variable 'y' equals 0 and is used as a divisor, causing division by zero.
8	19	Array 'arr' has 4 elements (valid indices 0–3); accessing arr[4] is out of bounds.
9	20–21	Pointer 'ptr' is dereferenced without being initialized to a valid address.
10	23	The expression contains a redundant '+ 0' operation that does not change the result.
11	24–26	Variable 'x' is never updated inside the while loop, causing an infinite loop.

(b) Classify by Compiler Phase

Category	Line(s)	Explanation
A. Lexical Error	8	'#' is not a valid character in a C identifier.
B. Syntax Error	4, 9, 13	Each statement is missing its terminating semicolon.
C. Semantic Error	11, 12	Line 11: argument-count mismatch in the call to divide(). Line 12: '=' is legal C syntax but semantically wrong here — it assigns 5 to result and evaluates the whole expression as true, instead of comparing result to 5.
D. Intermediate Code Issue	23	The compiler evaluates 'y * result' before adding 'x', per standard operator precedence; if the programmer intended a different grouping, the generated three-address code (t1 = y*result; t2 = x+t1) will not match their expectation, producing a silently wrong value.
E. Optimization Issue	23	The sub-expression '+ 0' is redundant and can be eliminated by constant folding without changing the result — an unoptimized compiler leaves this dead computation in the generated code.

F. Runtime / Target Code Error	17, 19, 20–21, 24–26	Division by zero (17), array bounds violation (19), uninitialized pointer dereference (20–21), and an infinite loop (24–26) all manifest only when the target code actually executes.
--------------------------------	----------------------	---

(c) Compilation vs Execution (Evaluation)

Detected at compile time: The illegal symbol '#' on line 8 (lexical), the missing semicolons on lines 4, 9, and 13 (syntax), and the argument-count mismatch on line 11 (semantic, since the prototype of divide() is visible to the compiler) are all caught before the program can run. The assignment-in-condition on line 12 is legal C, so most compilers only issue a warning such as 'suggest parentheses around assignment used as truth value' rather than a hard error.

Detected only during execution: Division by zero (line 17), the array-bounds violation (line 19), the uninitialized-pointer dereference (lines 20–21), and the infinite loop (lines 24–26) are all syntactically and semantically legal C constructs. The compiler has no way to know the runtime values of y, the array index, or the pointer, so these problems only surface as abnormal termination, memory corruption, or a hang once the program is executed.

Why some errors prevent further compilation: The lexical error on line 8 and the missing semicolons corrupt the token stream / parse tree at the point they occur. Since every later phase

— semantic analysis, intermediate code generation, optimization, and target code generation
— operates on the output of the previous phase, the compiler cannot reliably continue past a broken parse tree. It therefore enters error-recovery mode (skipping to the next synchronizing token such as a semicolon or brace) purely to keep searching for more errors, but it will not proceed to generate any executable code until the source is syntactically valid.

(d) Correct the Program (Application)

The fully corrected program below fixes every identified error while preserving the original intent of the code: it uses proper syntax, correct semantics (comparison instead of assignment, the correct number of function arguments), a non-zero divisor, an in-bounds array access, a validly initialized pointer, a simplified expression, and a loop that terminates safely.

```
#include <stdio.h>
int divide(int a, int b)
{ return a / b;
}
int main() {
    int value = 20;
    float num = 15.5;
    int result;
    result = divide(10, 2);
    if (result == 5) {
        printf("Result is 5\n");
    }
    int x = 10;
    int y = 2;
    int z = x /
    y;
    int arr[4] = {1, 2, 3, 4};
    printf("%d\n",
    arr[3]); int target = 0;
```

```

int *ptr = &target;
*ptr = 25;
int total;
total = x + y * result;
while (x < 20) {
    printf("%d\n", x);
    x++;
}
return 0;
}

```

Expected Output:

Result is 5

```

4
10
11
12
13
14
15
16
17
18
19

```

(e) Deep Analysis (Evaluation)

Incorrect pointer usage: Writing through an uninitialized pointer, as in `*ptr = 25` on lines 20–21, stores 25 at whatever arbitrary address happens to be in `ptr` at that moment. Because C performs no automatic validity checking on pointers, this can silently overwrite unrelated variables, corrupt the call stack or heap, or immediately crash the program with a segmentation fault. The unpredictability of the address makes this class of bug extremely difficult to reproduce and debug, and it is one of the most common root causes of security vulnerabilities in C programs.

Array bounds violation: `arr[4]` on line 19 reads one element past the end of a 4-element array (valid indices 0–3). C arrays carry no built-in bounds information at runtime, so the read simply returns whatever value is stored in the adjacent memory location — this could be another local variable, saved register data, or unallocated memory. The impact ranges from an incorrect but 'plausible-looking' output to a crash, and in the case of writes (rather than reads) it can corrupt program state or be exploited as a buffer-overflow vulnerability.

Operator precedence and intermediate code: In the expression `'x + y * result + 0'` on line 23, the compiler applies the standard C precedence rules (multiplication before addition) regardless of how the programmer visually grouped the terms. The intermediate code generator therefore

produces three-address code equivalent to 't1 = y * result; t2 = x + t1; total = t2 + 0;'. If a programmer mistakenly assumed left-to-right evaluation (as if it were '(x + y) * result'), the compiled program computes a completely different, silently wrong value — illustrating why understanding precedence is essential when reasoning about how source expressions become intermediate code.

(f) Advanced Task (Create Level)

One improvement would be to add a data-flow / definite-assignment analysis pass right after semantic analysis, one that tracks whether every pointer has been assigned a valid address and every array index falls within statically known bounds before it is used. This would let the compiler flag line 17 (a divisor that is a compile-time-known zero), line 19 (an index that is out of range at compile time) and lines 20–21 (a pointer that was never assigned) as warnings or errors up front, instead of leaving them to be discovered only when the program actually runs.

A second improvement would be to make the diagnostic messages explain why something is wrong and how to fix it, not just what rule was broken. For line 12, for instance, the compiler could say something like: assignment used as a condition here — did you mean "=="? this always evaluates to true and also changes the value of "result". Separating genuine errors from likely-bug warnings, and pointing to the exact column in the source, would make debugging far less frustrating.

As for the optimization phase, it can be taught to look for simple identities such as $\text{expr} + 0$ and $\text{expr} * 1$ and simplify them away automatically through constant folding, along with common sub-expression elimination so that a value like $y * \text{result}$ is computed once and reused instead of being recalculated. Applying this to line 23 would remove the redundant "+ 0" without changing what the program actually does.